

quickstart

Quickstart — Workshop Curriculum Platform (Phase 1 validation guide)

Feature: specs/001-workshop-curriculum-platform | **Date:** 2026-09-01
Reads with: [spec.md](#) · [plan.md](#) · [research.md](#) · [data-model.md](#) · contracts/

This is a **validation guide**, not an implementation guide. It says how to *run* the feature and how to *observe* whether each success criterion actually holds. It restates neither the interface shapes (see contracts/) nor the entities (see [data-model.md](#)).

Every path in this document is **relative to the repository root** (the directory containing workshop/, scripts/ and specs/). Run every command from there unless a step says otherwise.

How to read this document

Nothing below is presented as proven unless it was actually run. Each step carries one status:

Marker	Meaning
RUNNABLE NOW	The command was executed while authoring this guide, or it invokes only tooling verified present in Preflight .
BLOCKED: BUILD	The component does not exist yet. The command shown is the contract this scenario expects ,

Marker	Meaning
	not a tested invocation. It has never been run.
BLOCKED: OPERATOR	Needs a human action outside this repository — installing an ASR engine, pulling a generative model.
BLOCKED: U1...U5	Blocked on a named open item from <u>research.md</u> § <i>Open items carried into Phase 1</i> .

A step may carry more than one block. Where a step is blocked, the **expected observable outcome** is still written out, because that is the acceptance target the implementation is aiming at — it is just not evidence yet.

Status at a glance

Scenario	Proves	Executable today?
<u>0. Preflight</u>	environment truth	Yes, entirely. Three of its six checks are expected to report 1 FAIL on this host.
<u>US1 reassemble</u>	FR-007	Yes. Verified tooling, with an existing paired mutation proof.
<u>US1 transcribe → SC-001/002/003</u>	SC-001, SC-002, SC-003	No. BLOCKED: OPERATOR (no ASR engine) + BLOCKED: BUILD + U1, U2, U3.
<u>US2 browse and watch</u>	SC-004	No. BLOCKED: BUILD only — no operator action needed once built.
<u>US3 search</u>	SC-005, SC-006, SC-007, SC-008	No. BLOCKED: BUILD; SC-006 additionally at risk (U4), SC-007/SC-008 additionally BLOCKED: U5.
	SC-009, SC-010	

Scenario	Proves	Executable today?
<u>US4 answers — config A extractive</u>		No, but only on BUILD. Needs no generative model. This is the path that can be validated on this host.
<u>US4 answers — config B generative</u>	SC-009, SC-010 under generation	No. BLOCKED: OPERATOR (ollama pull) + BLOCKED: BUILD.
<u>US5 add a chapter</u>	SC-011, FR-026, FR-027, SC-016	No. BLOCKED: BUILD; the end-to-end variant inherits the ASR block.
<u>Governance gates</u>	SC-014, FR-034, FR-035	Yes, entirely.
<u>Evidence and mutation proofs</u>	SC-012, SC-013, SC-018	Partly. The two gates that ship - -prove-failure today are runnable; the feature's own gates are BLOCKED: BUILD.

One sentence of honesty up front: on this host, today, the only parts of this feature that can be executed end to end are the recording reassembly, the governance gates, and the mutation proofs of two existing scripts. Everything else is waiting on either code that has not been written or a tool that has not been installed. This document names which, per step, rather than implying a working system.

0. Preflight — verify the environment before anything else

Run this first, always. Several assumptions that this feature's request rested on are **false on this host**, and each one fails in a way that looks like something else — a missing transcoder reports a bad-argument error, a missing speech recogniser reports a working binary of the same name. Failing loudly here is the entire point of the section.

The reporting contract

Every check reports one of exactly three codes. This is the spec's FR-033 / SC-013 requirement applied to the preflight itself, and it mirrors the 0 / 1 / 2 convention that `scripts/verify-governance-cascade.sh` already uses in this repository:

Code	Name	Meaning
0	PASS	Positive evidence was obtained that the capability exists <i>and behaves as required</i> .
1	FAIL	Positive evidence was obtained that it is absent or wrong.
2	UNDETERMINED	The check itself could not run. Never a pass, never a fail.

“Could not determine” is not a pass. A preflight that cannot see must say it cannot see.

P1 — `ffprobe` (expected: 1 FAIL on this host)

RUNNABLE NOW

Why the glob. `workshop/chapters/01/*Recording.mp4` is written as an unquoted glob rather than a literal path because the literal name carries a third party's first name. The glob is not a loss of precision: that directory holds exactly one `*Recording.mp4`, and a glob expansion is not word-split, so the embedded spaces are handled correctly. Same for `*Recording.mp4.sha256` in US1 below. See [docs/content-boundary-incident-2026-09-01.md](#) §11.

```
command -v ffprobe && readlink -f "$(command -v ffprobe)"
ffprobe -v error -show_format \
  workshop/chapters/01/*Recording.mp4 \
  >/dev/null; echo "rc=$?"
```

Observed on this host (2026-09-01):

```
<home>/bin/ffprobe
<home>/cache/ms-playwright/ffmpeg-1011/ffmpeg-linux
Unrecognized option 'show_format'.
```

Error splitting the argument list: Option not found
rc=8

Verdict: 1 FAIL. `ffprobe` is a **symlink to Playwright's bundled ffmpeg binary**. It is not `ffprobe`. It accepts `-version` — which is why an earlier probe using `ffprobe --version` concluded it was present — and rejects `-show_format`, the option every media-inspection step in this pipeline actually needs.

The trap, stated so it is not re-sprung: probing with `--version` proves a binary answers, not that it is the right binary. The check **MUST** exercise the option the pipeline depends on. A preflight that runs `ffprobe --version` reports 0 PASS here and is wrong.

Consequence for the run guide: any step below that would call `ffprobe` is marked **BLOCKED: OPERATOR**. Duration, codec and stream properties for Chapter 1 are **not** re-derivable on this host today; the values quoted in research.md (6928.75 s, H.264 1080p24, AAC-LC 48 kHz, dual-mono) were measured elsewhere and are carried, not reproduced.

P2 — ffmpeg (expected: 1 FAIL as a durable dependency)

RUNNABLE NOW

```
command -v ffmpeg && readlink -f "$(command -v ffmpeg)"
ffmpeg -version | head -1
case "$(readlink -f "$(command -v ffmpeg)")" in
  *ms-playwright*) echo
    "FAIL: ffmpeg resolves into an npx-managed Playwright
    cache" ;;
  "")
    echo "UNDETERMINED: could not resolve ffmpeg" ;;
  *)
    echo "PASS: ffmpeg is durably installed" ;;
esac
```

Observed on this host (2026-09-01):

```
<home>/bin/ffmpeg
<home>/cache/ms-playwright/ffmpeg-1011/ffmpeg-linux
ffmpeg version 7.0.2-static https://johnvansickle.com/ffmpeg/ ...
FAIL: ffmpeg resolves into an npx-managed Playwright cache
```

Verdict: 1 FAIL as a pipeline dependency, even though the binary works right now. It lives in `~/cache/ms-playwright/ffmpeg-1011/`. A `npx playwright uninstall`, a cache clean, or a Playwright version bump

removes it. A content pipeline that runs a workshop curriculum must not depend on a browser-automation cache — that is a dependency nobody declared and nobody maintains.

The check distinguishes the two failure modes deliberately: an *unresolvable* ffmpeg is 2 (the instrument could not see), a *cache-resident* ffmpeg is 1 (it saw, and the answer is bad).

P3 — speech-recognition engine (expected: 1 FAIL on this host)

RUNNABLE NOW

```
command -v whisper && rpm -qf "$(command -v whisper)" 2>/dev/null
python3 -c "import whisper" 2>&1 | tail -1
python3 -c "import faster_whisper" 2>&1 | tail -1
```

Observed on this host (2026-09-01):

```
/usr/bin/whisper
whisper-1.3.1-alt1.noarch
ModuleNotFoundError: No module named 'whisper'
ModuleNotFoundError: No module named 'faster_whisper'
```

Verdict: 1 FAIL. No speech-recognition engine is installed. /usr/bin/whisper is whisper-1.3.1-alt1, an unrelated microphone-loopback GUI by a different author. It shares a name with OpenAI Whisper and nothing else.

The trap: `command -v whisper` succeeds. A preflight that stops there reports 0 PASS and is wrong. The check MUST resolve the package identity *and* attempt the import that the pipeline would actually perform.

Note also that ollama's /v1/audio/transcriptions endpoint exists but does not close this gap — per [research.md](#) D-TRANS-1 its Whisper strings are an *encoder*, with no `word_timestamps`, `avg_logprob` or `no_speech_prob`, so it cannot satisfy FR-002 or FR-003. Do not treat the presence of that endpoint as a passing ASR check.

Consequence: US1 step 2 onward is BLOCKED: OPERATOR.

P4 — generative model (expected: 1 FAIL on this host)

RUNNABLE NOW — read-only; it does not load a model and does not restart the service.

```
curl -s --max-time 8 http://localhost:11434/api/tags |  
python3 -c 'import json,sys; d=json.load(sys.stdin);  
[print(m["name"], m["details"]["family"], m["details"]  
["parameter_size"]) for m in d["models"]]'
```

Observed on this host (2026-09-01):

```
jina-embeddings-code-cpu:latest          jina-bert-v2  160.28M  
ordis/jina-embeddings-v2-base-code:latest jina-bert-v2  160.28M
```

Verdict: 1 FAIL. There is no generative model. Both entries are the *same* embedding model — jina-bert-v2, an encoder with no LM head. ollama here serves embeddings and nothing else.

The check must classify by **family and role**, not by count. “Two models are present” is true and useless; “zero decoders are present” is the fact that matters.

Consequence: US4 config B is BLOCKED: OPERATOR. US4 config A is not — it needs no generative model at all, which is precisely why it is the default working path (D-LLM-5).

P5 — container runtime (expected: 0 PASS for podman, docker absent)

RUNNABLE NOW

```
command -v podman && podman --version  
podman info --format '{{.Host.Security.Rootless}}' 2>/dev/null ||  
    echo "UNDETERMINED: podman info failed"  
command -v docker || echo "docker: ABSENT (expected)"
```

Observed on this host (2026-09-01): podman version 5.7.1; docker is **not on PATH**.

Verdict: 0 PASS for the runtime the plan targets. §11.4.161 mandates rootless podman and this host has it. Docker’s absence is **expected and correct**, not a failure — any script in this feature that invokes docker is a portability defect, and scripts/audit-environment-assumptions.sh is the gate that catches it.

If `podman info` fails (for example, no user namespace mapping), report 2 UNDETERMINED — do not infer rootlessness from the version string.

P6 — local gate suite reachable

RUNNABLE NOW

```
bash scripts/pre-push-gates.sh --list
```

Observed on this host (2026-09-01) — 8 gates, E and 0–6:

```
ID  GATE
COMMAND
E    $11.4.156(E) no active root CI config                                git
ls-files | grep -E '^\.github/workflows/.*\.ya?ml$|^\.gitlab-
ci\.yaml$'    (must be EMPTY)
0    hardcoded path audit                                                ./
scripts/audit-hardcoded-paths.sh
1    Go unit tests (_tools/gen)                                          cd
_tools/gen && go test ./...
...
6    Playwright (chromium), excluding the all-language crawl cd
_tests && npx playwright test --project=chromium --grep-invert
'all-language'
```

Verdict: 0 PASS — the runner exists and can enumerate its own gates.

Not a pass for enforcement. `.git/hooks/` is untracked, so a fresh clone runs **no** gates until `bash scripts/pre-push-gates.sh --install` has been run, and `git push --no-verify` bypasses the hook with no record. Verify installation separately: `ls -l .git/hooks/pre-push`. Absent $\Rightarrow$ 1 **FAIL** for enforcement, regardless of P6 passing.

Preflight summary for this host

Check	Code	One-line reason
P1 ffmpegprobe	1 FAIL	symlink to Playwright's ffmpeg; rejects <code>-show_format</code> (<code>rc=8</code>)
P2 ffmpeg	1 FAIL	works, but resolves into an npx-managed cache — not a durable dependency
P3 ASR engine	1 FAIL	<code>/usr/bin/whisper</code> is an unrelated GUI; both Python imports fail
P4 generative model	1 FAIL	two models, both the same 160 M embedding encoder

Check	Code	One-line reason
P5 podman rootless	0 PASS	podman 5.7.1; docker correctly absent
P6 gate suite	0 PASS (runner)	8 gates enumerable; enforcement is a separate, per-checkout check

A preflight that reports anything other than four 1s and two 0s on this host has not understood the host. If a future run reports 0 PASS for P1, P3 or P4, the first hypothesis is that the *check* regressed, not that the environment improved — verify the operator actually installed something before believing it.

US1 — Read what was actually said: the transcript

Proves: SC-001 (100% coverage), SC-002 (accuracy measured over ≥ 30 sampled windows), SC-003 (timestamps within 5 s). Requirements FR-001...FR-007.

US1 step 1 — reassemble the recording from its 36 parts

Status: **RUNNABLE NOW.** This is the one part of US1 that works today, on tooling that already exists and already ships its own mutation proof.

Prerequisites: none beyond a checkout. No ASR, no ffmpeg, no containers. The reassembler uses sha256sum, cat and tar only.

```
# Confirm what is actually committed: 36 parts + one manifest,
# video git-ignored.
ls workshop/chapters/01/*.tar.gz.part-* | wc -l
head -5 workshop/chapters/01/*Recording.mp4.sha256

# Reassemble, hash-verified at every stage. Time it – SC-004 needs
# this number.
time bash workshop/scripts/extract-videos.sh
```

Expected observable outcome

ls | wc -l prints 36. The manifest's first five lines are # video-archive-manifest v1, video_sha256=345a7439..., video_size=1871981557, archive_sha256=67ee3b72..., part_size=50m — *verified 2026-09-01.*

`extract-videos.sh` prints one of exactly two things per manifest, and never anything ambiguous:

- `[extract] skip (intact): .../Recording.mp4` — the video is already present and its hash matches;
- `[extract] extracting: ... then [extract] ok: .../Recording.mp4 (36 part(s) verified)`.

It exits 0. Any hash mismatch exits non-zero with `[extract] ERROR: ...` naming the failing stage (hash mismatch on part, archive stream hash mismatch, or extracted video hash mismatch).

Success criterion proved: FR-007 — automatic reassembly with integrity verification that fails loudly. Four independent hash checks run: per-part (36), reassembled stream, and the extracted file against the manifest.

Its paired mutation proof already exists (this is the FR-032 / SC-012 half, and it predates this feature):

`bash workshop/scripts/self-test.sh`

Described rather than quoted — **the private workshop carriers' own wording of what this script does is private content and is not reproduced here**; read it in that submodule at `AGENTS.md` (and its three sibling carriers). Independently: the script generates throwaway input, runs the archive and extract halves and checks that the round trip reproduces the input bit-for-bit, and then **damages one part on purpose and requires the extractor to reject it**. The seeded-damage half is the §1.1 paired mutation: a green round trip on undamaged input is not evidence on its own, because a reassembler that never rejects anything would also produce it. The refusal is what carries the proof.

Timing, honestly. Measured on this host on 2026-09-01: `sha256sum` on one 52,428,800-byte part took **0.35 / 0.38 / 0.40 s** warm (≈ 138 MB/s), at load average 5.30 on 8 CPUs. The extractor hashes ≈ 5.4 GB in total (36 parts + the reassembled stream + the 1,871,981,557-byte output), then `gunzip`s ≈ 1.79 GB and writes ≈ 1.87 GB. **Do not quote a predicted wall time from this** — cold I/O dominates and this host was under load. Record the actual time output; it is an input to SC-004.

US1 step 2 — transcribe (BLOCKED)

Status: BLOCKED: OPERATOR (P3, no ASR engine) + BLOCKED: BUILD (the pipeline does not exist) + BLOCKED: U1, U2, U3.

Do not run a transcription pipeline from this guide. There is nothing to run, and the engine it would need is not installed. The command shape below is the **contract this scenario expects**; it has never been executed.

Prerequisites that must be satisfied first, in order:

1. **Operator:** install faster-whisper (CTranslate2, CPU int8, large-v3-turbo) into a project-local virtual environment — decision D-TRANS-1. This is an operator action, outside this repository, and it is **not** something this guide performs.
2. **U1** — whether the cp314 wheels for CTranslate2/faster-whisper install and run on this host (Python 3.14.6 is what is present) is **unknown**. It closes by attempting the install.
3. **U2** — real transcription throughput on this CPU is **unknown**. It closes with a 5-minute calibration run on an already-extracted 300 s sample.
4. **U3** — **nobody has listened to the recording and no ASR has run on it**. That the recording contains intelligible speech in a known language is an assumption, not a finding. It closes in the same calibration run.

U1, U2 and U3 all close in that one short calibration. It is the single recommended next action for this user story, and it must happen before any transcription estimate in this document is treated as a plan.

```
# CONTRACT ONLY – never executed. See contracts/ for the
# authoritative shape.
bash workshop/pipeline/transcribe/run.sh --chapter 01 --calibrate
300 # U1+U2+U3
bash workshop/pipeline/transcribe/run.sh --chapter
01 # full run
```

Expected observable outcome (the acceptance target, not evidence)

- A markdown transcript at workshop/chapters/01/transcript/, organised into navigable sections (FR-011), each passage carrying pid, t_start_s, t_end_s, provenance, confidence and uncertain per [data-model.md](#).
- Passages the engine was not confident about are **marked**, not filled in (FR-003).

- speaker is null unless a human attributed it — never machine-guessed. The audio is dual-mono (L-R difference -90.3 dB) and uniformly AGC-compressed, so both cues diarization needs are measurably absent (D-TRANS-2). A transcript that arrives with machine-assigned speakers is a defect, not a bonus.
- Progress is reported and the run is resumable: atomic checkpoint per ≤ 300 s chunk, cut inside measured silence, `condition_on_previous_text=False` (FR-029, D-TRANS-4). Killing it mid-run and restarting must not lose completed chunks — **that interruption test is itself the mutation proof for FR-029** and must be run, not assumed.

Timing, honestly. The estimate is **40–80 minutes idle** for the 01:55:28.75 recording, and **roughly double under load** — this host sat at load average 5.30/8 CPUs while this guide was written, with a `lumen index --force rebuild` in flight. That estimate is **UNVERIFIED**; it is arithmetic from published throughput ratios, not a measurement of this machine. U2 exists precisely because nobody has measured it. Do not promise a number to a user until the calibration has run.

US1 step 3 — prove SC-001 (100% coverage)

Status: BLOCKED: BUILD (depends on step 2).

SC-001 is deliberately designed to be an **arithmetic identity, not a judgement** (D-TRANS-4). The check is:

```
U [t_start_s, t_end_s] over all transcript passages, unioned
with the VAD non-speech spans, equals [0, duration_s) exactly,
with duration_s = 6928.75.
```

CONTRACT ONLY — never executed.

```
bash workshop/pipeline/transcribe/verify-coverage.sh --chapter 01
```

Expected observable outcome: a report stating covered seconds, non-speech seconds, and `gap_seconds = 0.000`, or an enumeration of every gap with its start and end. Anything else — a percentage without the underlying spans, a “looks complete” — is not evidence and does not satisfy SC-001’s “every gap explicitly accounted for”.

Why this design matters: the 8 measured long silences totalling 41.33 s (0.597% of the recording) are a **free test fixture** for the VAD side of the identity. If the coverage checker reports zero non-speech spans, the checker is broken, not the recording.

Paired mutation proof (FR-032): delete one passage from a copy of the transcript and re-run. The checker **MUST** report the resulting gap and exit non-zero. A coverage checker that still says `gap_seconds = 0.000` after a passage is removed is measuring nothing.

US1 step 4 — prove SC-002 (measured accuracy, ≥ 30 sampled windows)

Status: BLOCKED: BUILD + blocked on 1–2 hours of human transcription.

This is the load-bearing measurement in the whole feature, and it is the one people are most tempted to fake. Read D-TRANS-3 before running it.

The sampling unit is the audio timeline, not the machine's own passages. Word error rate is computed against a **blind human reference** over ≥ 30 seeded, stratified 30-second audio windows.

Why not sample passages — and this is the point that must not be lost: sampling machine passages makes whole-region deletions *structurally invisible*. A span the recogniser dropped produces no passage, so it can never be sampled, and accuracy is biased upward precisely where the transcript is worst. Sampling the timeline cannot miss them. Thirty 30 s windows span well over 30 passages, so SC-002's own wording is satisfied while the bias is removed.

```
# CONTRACT ONLY — never executed.
bash workshop/pipeline/transcribe/sample-windows.sh --chapter 01
--n 30 --seed <recorded>
# → human transcribes the 30 windows BLIND (without seeing machine
output). 1–2 hours.
bash workshop/pipeline/transcribe/measure-accuracy.sh --chapter
01 --reference <path>
```

Expected observable outcome: a verification report published **alongside the transcript** (FR-004) stating the measured WER, the number of windows, the seed, and the **hash of the normaliser, frozen before the first measurement**. Companion metrics in the same report: coverage-gap rate (SC-001), timestamp error median and p95 against the 5 s bound (SC-003), and speaker attribution accuracy (FR-005).

Success criterion proved: SC-002, and only if all four of these are true — otherwise the number is decoration:

1. the human reference was produced **blind**;
2. the seed is recorded, so the sample is re-drawable;
3. the normaliser was hashed and frozen **before** the first measurement, not tuned afterwards;
4. the report states the method, not just the figure (FR-004 requires the method).

Paired mutation proof: run the scorer against a reference that has had 10% of its words corrupted. The reported WER **MUST** move by a comparable amount. A scorer that returns a similar figure regardless of the reference is not measuring accuracy.

US1 step 5 — prove SC-003 (timestamps within 5 s)

Status: BLOCKED: BUILD (depends on step 2, and shares step 4's sampled windows).

```
# CONTRACT ONLY – never executed.  
bash workshop/pipeline/transcribe/measure-timestamps.sh --chapter  
01 --reference <path>
```

Expected observable outcome: for each of the ≥ 30 sampled windows, the absolute difference between the passage's `t_start_s` and the human-marked onset of that speech. The report states median and **p95**, and the pass condition is the SC-003 bound: a reader lands **within 5 seconds** of the spoken content.

Report the **distribution**, not a mean. A mean hides the tail, and the tail is where a reader lands on the wrong sentence.

Manual cross-check a reviewer can perform without any tooling — the spec's own Independent Test for US1: pick five passages at random, open the reassembled `Recording.mp4` in any player, seek to each stated timestamp, and confirm the words match. Five hand-checks that disagree with a green automated report mean the report is wrong.

Success criterion proved: SC-003.

US2 — Browse and watch the curriculum

Proves: SC-004 (fresh clone to running in under 15 minutes).
Requirements FR-008...FR-013.

Status: **PARTIALLY RUNNABLE (updated 2026-09-01)**. ~~BLOCKED:—BUILD.~~ ~~The platform does not exist yet.~~ That status is **withdrawn** — the control plane now exists and runs. Measured on this host at 2026-09-01, not read off a document:

```
podman ps → workshop-curriculum_platform_1    Up (healthy)
curl /api/health →
{"status":"ok","pid":1,"chapters":1,"web":false,
  "ollama":{"configured":true,"reachable":true}}
```

Read the boundary precisely, because “runnable” is not “US2 passes”. What is measured: the container stack starts, reports healthy, and serves the API. What is **not** measured and must not be inferred: “web”: false in that same response says the browsing frontend is **not being served yet**, so outcomes 2–4 below (chapter list, inline playback, position and progress) remain ~~BLOCKED: BUILD.~~ The SC-004 end-to-end clock has also not been run from a genuinely fresh clone, which is the only measurement that proves SC-004. Nothing here claims it has.

Still not blocking, as before: no ASR engine, no generative model, and no open research item.

Prerequisites: a fresh clone; Go 1.26 (verified present: go1.26.2); Node 20+ (verified present: v22.19.0) if the frontend is built rather than shipped prebuilt.

The reference control plane is `ai_interviewing/platform/scripts/`, which already has `{build,ingest,restart,start,status,stop}.sh` — FR-013 requires a person familiar with one module to navigate the other, so the workshop’s script **names** match.

Path corrected 2026-09-01. An earlier revision of this guide invoked `workshop/platform/scripts/...`, mirroring the reference’s directory nesting. That path is **withdrawn**: the contract settles the control plane at `workshop/scripts/`, alongside the pre-existing `extract-videos.sh / archive-videos.sh / self-test.sh / install-hooks.sh` that FR-007 already depends on. One script directory per module — a split would leave two with no rule for which holds what, which serves FR-013 worse than a structural echo serves it well. See [contracts/pipeline-](#)

[cli.md §2.1](#) for the full decision. Note also what the old text got right and what it did not: the *names* were always correct; the *directory* was the mirror carried one level too deep.

```
# Time the WHOLE block; SC-004 is a clock, not a feeling.
git submodule update --init workshop           # or a fresh clone
bash workshop/scripts/extract-videos.sh        # RUNNABLE NOW –
    see US1 step 1
bash workshop/scripts/start.sh                 # RUNNABLE NOW –
    brings up the container stack
# → open the URL the start script prints (the API; the browsing UI
    is not served yet)
bash workshop/scripts/status.sh                # RUNNABLE NOW
bash workshop/scripts/stop.sh                  # RUNNABLE NOW
```

`start.sh` brings up a **container stack**; it is not a native process launch. The seam is the workshop-boot adapter over submodules/containers' `pkg/compose / pkg/health / pkg/runtime` Go API — **not** that submodule's `cmd/boot` CLI, which was measured unusable for this workload (no `compose-file` flag, no lifecycle verbs, a hardcoded helixagent endpoint, and an exit 0 after starting nothing). [contracts/pipeline-cli.md §4.8.0](#) records that measurement in full; the short version is that a wrapper trusting `cmd/boot`'s exit code would report a healthy stack that does not exist.

`start.sh` builds on demand, so a fresh clone needs no separate build step — which is what makes the SC-004 15-minute budget a single block rather than a sequence.

One host note, so a reader is not surprised: the contract's default port is 8080, and this host is currently running the stack on 8087 via the documented `WORKSHOP_PORT` override. Read the actual address from `workshop/platform/run/server.json` rather than assuming either number.

Expected observable outcome

1. `start.sh` prints UP: `<url>` and is **idempotent** — a second invocation prints already running – nothing to do and exits 0 **without recreating any container**, matching the reference module's behaviour in contract if not in mechanism. (*Corrected 2026-09-01: this line previously expected already running (pid ...). There is no host pid to print — the stack is containerised, and liveness is read from the container state and the resolved endpoint in `workshop/platform/run/server.json`, not from a pidfile. The idempotence guarantee is unchanged; the evidence behind it is.*)

2. The chapter list shows **Chapter 1** with its title and a summary (FR-008). One chapter is the current reality; a list that only works with several is not tested by this.
3. Opening Chapter 1 shows its transcript and the notes PDF, and **plays the recording inline** (FR-009). The user is never asked about parts, archives or tar — reassembly is the platform’s problem (FR-007, decision D3: local file, no streaming server, no transcoding, no CDN).
4. Navigating between sections and returning preserves position and progress (FR-010).
5. `stop.sh` exits 0 and `status.sh` then reports not-running.

Success criterion proved: SC-004 — fresh clone to running curriculum in **under 15 minutes**, using only this guide.

The honest part of SC-004. The 15-minute clock **must include the reassembly** in step 2 — ≈ 5.4 GB of hashing plus ≈ 1.79 GB of gunzip plus a ≈ 1.87 GB write. Measured on this host, `sha256sum` ran at ≈ 138 MB/s warm under load average 5.30. A validation run that starts the clock *after* the video is already extracted has not measured SC-004; it has measured something easier. If reassembly turns out to consume most of the budget, that is a finding to report, not a reason to move the start line.

Paired mutation proof (FR-032): run `start.sh` with the reassembled video **absent**. It **MUST** report precisely what is missing and refuse, rather than starting a curriculum whose player silently 404s. “Starts anyway” is the failure this proof exists to catch.

Accessibility check (SC-017) — runs with US2 and US3

Status: BLOCKED: BUILD, but the *instrument* already exists in this repository, so it does not need to be invented: `@axe-core/playwright` (`^4.11.3`) is already a dependency of `_tests/`, and `_tests/evidence/ally-audit/run-audit.js` is the working precedent, with its JSON output retained under `_tests/evidence/ally-audit/axe-json/`.

Run an axe audit over the chapter list, a chapter page, and the search interface, and separately walk the whole search interaction **keyboard-only** — enter a query, move through suggestions, open a result — with no mouse. Expected outcome: **zero violations at Level A or AA**, and every search interaction completable from the keyboard (SC-017, FR-041, FR-042).

The spec calls out two specific failure points to test rather than assume: suggestions appearing as the user types **must be announced** to a screen reader, and **must not trap focus**.

US3 — Find anything by meaning

Proves: SC-005 (200 ms p95 suggestions), SC-006 (2 s p95 results), SC-007 ($\geq 90\%$ top-5 on ≥ 20 queries), SC-008 ($\geq 80\%$ on queries sharing no literal words). Requirements FR-014...FR-020.

Status: BLOCKED: BUILD for all four. SC-007 and SC-008 are **additionally BLOCKED: U5** — no corpus exists yet, so the benchmark set cannot be authored, let alone run.

Prerequisites: US1’s transcript exists and is ingested; the index has a live generation. Per D-SEARCH-2 there are **two search paths**, and conflating them is how SC-005 gets failed:

Path	Used for	Backing	Measured basis
lexical	type-ahead, per keystroke	SQLite FTS5 prefix index	p50 0.25 ms / p95 9.58 ms / p99 19.2 ms over 58,726 real symbols; index builds in 0.93 s / 15.9 MB
semantic	results, on submit	embedding vectors, 768- dim	query embed 18.2–21.0 s under load

US3 step 1 — SC-005, suggestions ≤ 200 ms p95

CONTRACT ONLY – never executed.
`bash workshop/scripts/bench-suggest.sh --keystrokes 500 --report <evidence-path>`

Expected observable outcome: a latency report with p50, **p95** and p99, measured **at the HTTP boundary** — not inside the FTS5 call. SC-005’s budget is 200 ms end to end; the measured FTS5 p95 of 9.58 ms leaves roughly 190 ms for HTTP, serialisation and paint, and it is that remainder where the budget is actually spent or lost.

Success criterion proved: SC-005, if and only if the harness measures the whole round trip.

Suggestions **must not** call the embedding path. SC-005's 200 ms budget is unreachable semantically — a single query embedding measured 18.2–21.0 s under load, two orders of magnitude over budget — and comfortable lexically. Suggestions therefore return *navigational targets*; meaning-based ranking happens on submit. A suggest endpoint that awaits an embedding fails SC-005 by ~100×, and no amount of tuning closes that.

US3 step 2 — SC-006, results ≤2 s p95

CONTRACT ONLY — never executed.

```
bash workshop/scripts/bench-search.sh --queries 20 --repeat 5 --  
report <evidence-path>
```

Expected observable outcome: p95 over the full submit path — retrieval, fusion of lexical and semantic, and response — under **2 seconds**. The report **MUST** also record **host load average** and **whether an index build was in flight**, because without those the number is not interpretable.

Success criterion proved: SC-006 — with a stated risk, recorded here rather than discovered later.

SC-006 is the criterion most at risk in this feature, and this guide will not pretend otherwise. Three identical two-word embed calls, model resident, minutes apart, measured **20.16 s / 11.05 s / 0.10 s** at load 8.25 — a **200× spread driven purely by queue contention**, not compute variance. ollama serves one queue and interactive queries contend directly with indexing. Two consequences for validation:

1. **A benchmark run taken while an index rebuild is in flight is not a valid SC-006 measurement.** That is exactly the condition on this host right now — a `lumen index --force` rebuild was running while this guide was written. Record the condition; do not average across it.
2. **U4 is open:** idle embedding latency has never been measured, because the rebuild has not finished. Until U4 closes, no claim about SC-006 is supportable in either direction — not “it passes”, and not “it cannot pass”.

The design mitigation is D-SEARCH-5's **reserved embedding capacity for interactive queries**. Without it, SC-006 fails on every chapter ingest. The benchmark must therefore be run **once while an ingest is deliberately in progress** — if the reserved capacity works, that run passes too; if it only passes on an idle machine, the reservation is not doing its job.

US3 step 3 — SC-007 and SC-008, retrieval quality

Status: BLOCKED: BUILD + BLOCKED: U5.

The benchmark is **≥ 20 meaning-based queries with known expected passages**, of which a subset **shares no literal words with its target**. Neither the queries nor the corpus exist. U5 closes only after U51 produces a transcript and it is ingested.

```
# CONTRACT ONLY – never executed. Neither the script nor the
benchmark file exists yet (T118, T064).
bash workshop/scripts/bench-retrieval.sh \
  --benchmark workshop/platform/backend/testdata/benchmark/
  retrieval.tsv --report <evidence-path>
```

BENCHMARK PATH CORRECTED 2026-09-01. ~~workshop/platform/qa/retrieval-benchmark.jsonl~~. Withdrawn, struck through rather than deleted. This guide and [tasks.md](#) named **two different paths in two different formats** for the same input — here `workshop/platform/qa/*.jsonl`, there `workshop/pipeline/benchmark/*.yaml` — and `ls` on 2026-09-01 showed **neither directory exists**. A third location shipped in the meantime and is wired to a working gate (`workshop/platform/backend/testdata/benchmark/questions.tsv`, read by `workshop/platform/backend/gates/bench-answers.sh`), so the canonical location is that one, TSV, for both benchmarks. The full reasoning and the counts measured in the shipped file are recorded once, in [tasks.md](#)'s File Structure section, rather than restated here.

Expected observable outcome: per-query top-5 hit/miss against the expected pid, and two headline figures:

- **SC-007:** $\geq 90\%$ of the ≥ 20 queries return the expected passage in the **top five**.
- **SC-008:** $\geq 80\%$ of the **zero-literal-overlap subset** succeed — the subset is what distinguishes meaning-based matching from keyword matching, and it must be identified in the benchmark file itself, verified by a mechanical check that the query and target

share no token after normalisation. An “obviously different wording” judged by eye is not a measurement.

Success criteria proved: SC-007, SC-008.

Paired mutation proof: run the benchmark against an **empty index generation**. Both figures **MUST** collapse to 0% and the run **MUST** exit non-zero. A benchmark that still scores well without an index is scoring the fixture, not the system.

US3 step 4 — the three-state contract (FR-019, FR-020)

Status: BLOCKED: BUILD. Listed separately because this repository has **already shipped this exact defect once**: `ai_interviewing/.../search.component.ts` renders backend errors as the empty state, so a saturated backend reads to the user as “nothing found”.

Three states must be distinguishable, end to end, in the UI and not merely in the API:

State	Shown as	Never shown as
results	ranked passages	—
no match	“nothing in the curriculum matches this” (FR-019)	unrelated results presented as answers
degraded / unavailable	“search cannot answer right now” (FR-020)	an empty result set

How to force each state for the test — all three must be *provoked*, not waited for:

1. a query with known results;
2. a query with known-zero results;
3. the backend made unavailable, plus the Lumen-specific case:
Lumen returns "No results found. | Warning: Index is being updated in the background..." as **one unstructured string**. The service **MUST** parse that and promote it to **Degraded** — forwarding it as “no results” is the failure. Test with that exact string.

Do not gate degradation on a health probe. Measured in a single MCP session: `health_check` returned `Status: OK / service is healthy` in 2 ms, and seconds later `semantic_search` failed with all embedding

servers exhausted ... context deadline exceeded. It is a liveness probe, not a saturation probe. The three-state verdict must be derived from **the search call's own result** (D-SEARCH-4).

US4 — Ask a question and get a grounded answer

Proves: SC-009 (100% of citations genuinely support their claim), SC-010 (10/10 unanswerable questions declined). Requirements FR-021...FR-025.

There is no generative model on this host (preflight P4). This scenario is therefore written to run in **two configurations**, and the default one needs no generative model at all. A validation guide that could only be executed after an operator install would be untestable today, which is exactly the outcome to avoid.

Config A — the extractive adapter (the default working path)

Status: BLOCKED: BUILD only. No operator action, no model pull, no open research item. This is the configuration a reviewer can actually execute once the code exists, and it is the default per D-LLM-5.

```
# CONTRACT ONLY – never executed. See contracts/ for the Provider
  interface.
export WORKSHOP_ANSWER_PROVIDER=extractive
bash workshop/scripts/start.sh
bash workshop/platform/backend/gates/bench-answers.sh
# reads workshop/platform/backend/testdata/benchmark/questions.tsv
  by default;
# override with QUESTIONS=<path> and REPORT=<evidence-path>.
```

BENCHMARK PATHS AND SCRIPT LOCATION CORRECTED

~~2026-09-01. bash workshop/scripts/bench-answers.sh~~
~~answerable workshop/platform/qa/answerable-20.jsonl~~
~~unanswerable workshop/platform/qa/unanswerable-10.jsonl~~
~~report <evidence-path>~~ Withdrawn, struck through rather than deleted. Three things measured on 2026-09-01: workshop/
platform/qa/ **does not exist**; the harness shipped at workshop/
platform/backend/gates/bench-answers.sh, not under workshop/
scripts/; and it takes **one** file with a class<TAB>question schema
(A = expected answerable, U = expected unanswerable) rather
than two, because a filename that encodes its own row count is

wrong the moment a row is added and two files can drift apart. Its QUESTIONS=/REPORT= environment overrides replace the --answerable/--unanswerable/--report flags this guide invented. **This is a path correction only** — the expected observable outcomes below are unchanged, and the shipped fixture holds **8 A rows against SC-009's ≥ 20** , so the command above is runnable but does **not** yet satisfy SC-009. See [tasks.md](#)'s File Structure correction for the full reasoning and T077 for the remaining work.

Expected observable outcome

- **≥ 20 answerable questions** each return status: answered, text, and **≥ 1 citation**. Zero citations while answered is **structurally undecodable** — the response schema sets "minItems": 1 — so it cannot occur, rather than being caught after the fact.
- **≥ 10 unanswerable questions** each return status: declined with a reason of below_threshold, margin_too_small or unsupported.
- Every response carries retrieval (top score **and margin**) **even on success**, so a 0.002-margin pass is visible as *fragile* rather than indistinguishable from a confident one.

Success criteria proved: SC-009, SC-010 — under extraction.

How SC-009 is actually established (attaching a citation is easy; proving it supports the claim is the work). Four layers, and any one failing refuses the **whole** answer — claims are never silently stripped:

Layer	Check	Determinism
L1	retrieval gate: min_score and min_margin	deterministic
L2	JSON-schema- constrained generation, "minItems": 1 on citations	structural
L3	every citation ID $\in$ the live generation's PID set	deterministic set membership
L4	support verification: embedding floor,	probabilistic

Layer	Check	Determinism
	then batched entailment	

SC-009 is unreachable without L3, and L3 is only possible because pid is a ULID minted at ingest rather than derived from content or position (D-SEARCH-1). This is why the passage-identity layer is a prerequisite for grounding, not a nicety.

A reviewer verifies SC-009 by hand on a sample of **at least 20 answers**: follow each citation to its passage and ask whether it supports the claim. Machine L3 membership proves the citation *resolves*; only human review proves it *supports*. Both are required by SC-009’s own wording (“verified by review of a sample of at least 20 answers”) — do not substitute one for the other.

How SC-010 is established, and the honest caveat. The ≥ 10 unanswerable questions are drawn from a 10-item **adversarial taxonomy** (D-LLM-3): near-miss attribute, false premise, uncomputable aggregate, misattributed speaker, lexically-overlapping-but-unanswerable, redacted passage, inaudible segment, and others. Ten astrophysics questions would pass any threshold and prove nothing; the near-miss case (“what Docker version did he say?”) is the one that scores high on similarity while being genuinely unanswerable, and the **margin** test is what catches it.

Two caveats that must travel with any SC-010 pass, and must not be dropped in a summary:

1. **10/10 at temperature 0 is *reproducible*, not *generalising*.** It must never be paraphrased as “the system never fabricates”.
2. **The extractive adapter cannot fabricate by construction** — it returns spans from the corpus. So a 10/10 under config A proves the **refusal plumbing and the calibrated thresholds** are correct. It does **not** exercise a generator’s tendency to invent, because there is no generator. Reporting config A’s 10/10 as evidence that generation is safe would be a bluff. Config B is what tests that, and config B cannot run here.

Timing, honestly: the extractive adapter answers in **~0.3 s**. That is the one place in this feature where “fast” is defensible.

Config B — a generative provider (blocked on an operator pull)

Status: BLOCKED: OPERATOR + BLOCKED: BUILD. Runs only after an operator installs a generative model. Nothing below has been executed and it must not be presented as though it had.

Operator prerequisite (an operator action, not a step this guide performs):

```
# OPERATOR ACTION — not run by this guide.
ollama pull <a decoder model>          # P4 currently reports zero
                                         decoders present
export WORKSHOP_ANSWER_PROVIDER=ollama  # or
                                         openai_compatible
```

Re-run preflight **P4** afterwards. It must flip from 1 FAIL to 0 PASS, classified by family and role — a second embedding model would not change the verdict and must not be allowed to.

Then run the **same** `bench-answers.sh` invocation as config A. The pass conditions are identical; what changes is that a real generator is now in the loop, so L2 and L4 are doing work they were not doing under extraction.

Expected observable outcome: identical verdicts, plus a **markedly different latency profile**. Estimated CPU-only generation, **UNVERIFIED**, with the method shown in [research.md](#) D-LLM-5: **~21 s (1.5 B) / ~42 s (3 B) / ~95 s (7 B) idle**, ×1.4 with verification, and worse under load. “Instant answers” is off by two orders of magnitude and no prompt engineering closes that gap — answering is asynchronous by design.

Enabling the GPU to close it is rejected outright. This repository already refused that bargain once, after a Vulkan fault silently corrupted the index; `library=cpu` and `GGML_VK_VISIBLE_DEVICES=-1` are deliberate.

The privacy negative control (FR-024)

Status: BLOCKED: BUILD, but it uses podman, which preflight **P5** confirms is present and rootless.

FR-024 forbids content leaving the machine when a local model is configured. A config flag is not a guarantee, so the validation is a **negative control** plus a packet capture (D-LLM-4):

1. resolved-address allowlist — `net.LookupIP + IsLoopback`, **not** hostname string matching, which is not a security boundary;
2. run the answering path inside an **egress-denied network namespace**, and assert that `curl https://example.com` from **inside that namespace FAILS**. That failing curl is what upgrades “we observed no egress” into “egress was impossible” — without it, the capture proves only that nothing happened to be sent;
3. packet capture asserting **zero non-loopback packets** across the full 20-answer and 10-refusal runs.

Success criterion proved: FR-024, and FR-023’s local-hosting half.

Answering unavailable (FR-025, SC-013)

Status: BLOCKED: BUILD. With `WORKSHOP_ANSWER_PROVIDER=none` — which is the **default for a fresh clone**, so it cannot leak and cannot bluff — asking a question must return `status: unavailable` with `reason.code: no_provider`, while **browsing and search continue to work**.

*(Corrected 2026-09-01 under the C4 arbitration. This line previously read `verdict: unavailable with refusal_reason: no_provider` — the worst surviving instance of the forked Answer contract, because an unavailable state carrying a **refusal** reason conflates the exact two states the paragraph below insists are distinct. `no_provider` is a cause of `unavailable`, never a reason for a decline, and it is not a member of the reason enumeration that declined draws from.)*

`unavailable` is a first-class third state, distinct from `declined`. Conflating them would make a missing provider look like a content gap — the same three-state discipline the preflight applies to itself. FR-025 is enforced by **separate route trees** (compile time), not a runtime check, so the test is that the browse and search routes are reachable and green while the answer route reports `unavailable`.

US5 — Add a chapter, and run it twice

Proves: SC-011 (a maintainer adds a chapter with zero code or config changes, under 30 minutes hands-on), FR-026, FR-027 (idempotency), FR-028, and SC-016 (cross-references survive correction and re-index).

Status: BLOCKED: BUILD. The end-to-end variant additionally inherits US1's ASR block (BLOCKED: OPERATOR, P3), because FR-026's procedure includes transcription.

Design note for contracts/, offered as an observation rather than a new requirement: if the ingest procedure accepts a **pre-supplied transcript fixture**, then the idempotency proof (FR-027 / SC-011) and the identity proof (SC-016) become runnable **without any ASR engine** — they are about ingest and identity, not about speech recognition. Without that seam, every part of US5 inherits the ASR block and none of it can be validated on this host. The seam is cheap and it is the difference between a testable and an untestable user story.

US5 step 1 — place materials and run the procedure

```
# CONTRACT ONLY – never executed.  
mkdir -p workshop/chapters/02  
# place the recording + materials per the documented procedure  
bash workshop/pipeline/ingest/add-chapter.sh --chapter 02
```

Expected observable outcome: the chapter is transcribed, published, indexed and cross-linked **with no code change and no configuration change** (FR-026). Chapter numbering is ordinal and zero-padded, following the existing chapters/01/ convention.

Success criterion proved: SC-011 — measure **hands-on** time (the maintainer's own minutes), not wall-clock. Transcription runs unattended for 40–80 minutes idle and roughly double under load; that is machine time, and SC-011's 30-minute budget is about the human. State both numbers in the report so the distinction cannot be blurred in either direction.

US5 step 2 — incomplete materials must be refused (FR-028)

```
# CONTRACT ONLY — never executed.
mkdir -p workshop/chapters/03
# deliberately missing the recording
bash workshop/pipeline/ingest/add-chapter.sh --chapter 03; echo
"rc=$?"
```

Expected observable outcome: non-zero exit, a message naming **precisely what is missing**, and **no chapter published**. A partially ingested chapter appearing in the chapter list as though it were complete is the failure this step exists to catch. Per [data-model.md](#), a chapter **MUST NOT** reach published with an incomplete material set.

Success criterion proved: FR-028. This step is *also* the paired mutation proof for step 1 — it is the demonstration that the ingest gate can fail.

US5 step 3 — run it twice (FR-027, the idempotency proof)

```
# CONTRACT ONLY — never executed.
bash workshop/pipeline/ingest/add-chapter.sh --chapter 02 # run
1
sha256sum workshop/curriculum/02/**/* > /tmp/run1.sums # or
the report's own digest
bash workshop/pipeline/ingest/add-chapter.sh --chapter 02 # run
2
sha256sum workshop/curriculum/02/**/* > /tmp/run2.sums
diff /tmp/run1.sums /tmp/run2.sums && echo "IDEMPOTENT"
```

Expected observable outcome: diff is **empty**. No duplicate passages, no second set of pids, no additional cross-references. Re-ingest is idempotent **on pid** — the ULID minted at first ingest is reused, which is what makes the second run a no-op rather than a near-copy.

Success criterion proved: FR-027, and the “no duplicate content” half of SC-011.

Compare artifacts by **content hash**, not by timestamps or file counts. Two runs that produce the same number of differently-worded files are not idempotent, and a file-count check would call them identical.

US5 step 4 — SC-016: cross-references survive correction and re-index

Status: BLOCKED: BUILD. This is the criterion the whole passage-identity design exists to satisfy, and it is the one that **fails invisibly** if it fails — links keep rendering, they just point at the wrong text.

```
# CONTRACT ONLY – never executed.
# 1. record every existing cross-reference and citation target
# 2. apply a human correction to a passage (a typo fix inside its
    text)
# 3. re-index
# 4. re-resolve every recorded reference
bash workshop/pipeline/crossref/verify-resolution.sh --before
    <snapshot> --after <snapshot>
```

Expected observable outcome: 100% of previously created cross-references and citations still resolve to the passage they originally referred to. Not “most”. Not “the ones that were checked”.

Why this is not optional, stated with the measurement behind it — a controlled three-run re-index measured Lumen’s chunk IDs directly:

Change	Observed
fix one typo inside a section, line numbers unchanged	ID changed 4b98e295c112a429 → 610e608a50f273a6
prepend a section, shifting every later section by 4 lines	all IDs unchanged ; line ranges moved

The IDs are content-derived and position-independent — one of the exact two forms FR-037 forbids, and the one that breaks on the operation this feature performs most often (FR-038 human corrections). Built on them, **SC-016 would score 0% for every corrected passage, silently**. Built on positional keys, run 3 shows those move too. Both available keys fail, in opposite directions. Hence: ULIDs minted at ingest, written back as `<!-- pid: ... -->` anchors, resolved through a committed registry whose `content_hash` column is **named so nobody mistakes it for identity**.

Paired mutation proof: deliberately swap the resolver to key on `content_hash`, re-run, and require the verifier to report **<100%** and exit non-zero. If it still reports 100%, the verifier is not checking what it claims to check — and this is precisely the mutation that would otherwise ship unnoticed.

Success criterion proved: SC-016.

US5 step 5 — search spans old and new chapters

Status: BLOCKED: BUILD. After Chapter 2 is ingested, re-run the US3 retrieval benchmark extended with queries whose targets are in Chapter 2. Results **MUST** span both chapters. A search that returns only the newest chapter, or only the first, has an index-generation bug that a single-chapter benchmark cannot see.

Governance — the gates that must be green

Status: RUNNABLE NOW, entirely. These are the only feature-adjacent checks that can be executed today in full.

Run all of these before considering any step above complete. Run them from the repository root.

```
bash scripts/pre-push-gates.sh # the 8-gate
                               suite: E, 0..6
bash scripts/verify-governance-cascade.sh
                               # carrier lockstep across the owned fleet
bash scripts/continuation-check.sh # CONTINUATION.md
                               in sync (§12.10, FR-035)
bash scripts/audit-hardcoded-paths.sh # no absolute
                               paths (incl. inside workshop/)
bash scripts/audit-environment-assumptions.sh # no undetected
                                               host assumptions
```

Expected observable outcomes

Gate	Pass looks like
pre-push-gates.sh	exits 0. bash scripts/pre-push-gates.sh --list enumerates 8 gates (E, 0–6) — <i>verified 2026-09-01</i> . Gate E is the §11.4.156(E) check: git ls-files \ grep -E '^\.github/workflows/.*\.ya?ml\$ ^\.gitlab-ci\.yaml\$' must return empty .
verify-governance-cascade.sh	exits 0 with FAIL count 0. It reports PASS / FAIL / NOTE / ENV and exits 0 / 1 / 2 — an ENV result is an <i>instrument fault</i> , not a tree violation, and must never be reported as a governance failure. exits 0. CONTINUATION.md is in sync (FR-035).

Gate	Pass looks like
continuation-check.sh	exits 0. Every script derives its root — <code>ROOT="\$ (cd -- "\$(dirname -- "\$ {BASH_SOURCE[0]})/.." && pwd)"</code> for bash, <code>path.resolve(__dirname, '..')</code> for node, <code>pathlib.Path(__file__).resolve().parents[N]</code> for python.
audit-hardcoded-paths.sh	
audit-environment-assumptions.sh	exits 0. Every host capability is detected , never assumed.

Success criteria proved: SC-014 (zero active server-side CI), **FR-034**, **FR-035**.

§11.4.156 — no CI may be added, including inside workshop/

This is not a style preference and it has no escape hatch. The anchor **names and refuses** the exemption vocabulary — there is no `--allow-ci`, no `--ci-exempt` — and a consumer carrier may only *extend* inherited rules, never weaken them. A project-local `Override §11.4.156` is structurally impossible, not merely disfavoured. **Do not propose one, and do not record this feature's container work as one.**

Concretely, for this feature:

- **No `.github/workflows/*.yaml`, no `.gitlab-ci.yaml`, anywhere in `workshop/`.** The umbrella's pre-push gate E was blind to submodules until it was fixed on 2026-09-01; it now derives the owned fleet from `helix-deps.yaml` and is mutation-proven against two submodules. So a CI file added inside `workshop/` **will** be caught — but the rule is the reason, not the gate.
- **if: false does not count as disabled.** Such jobs still queue provider-side runs.
- Enforcement is a **local pre-push hook**. `.git/hooks/` is untracked, so a fresh clone is unprotected until `bash scripts/pre-push-gates.sh --install` is run, and `git push --no-verify` bypasses it with no record. Verify:

```
ls -l .git/hooks/pre-push || echo "NOT INSTALLED – this checkout runs no gates on push"
```

Two governance facts about workshop/ worth recording, because the plan states one of them as still open. Measured 2026-09-01: workshop/ **now carries all four governance carriers** — CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md — so plan.md's "the workshop submodule currently carries **none** of the four carriers" is **out of date as a statement of the tree**. Whether scripts/verify-governance-cascade.sh now passes C1 and C6 on workshop is a separate question that this guide does **not** answer: run the cascade and read its own verdict. Do not infer a green cascade from the presence of four files.

Second: §11.4.76 requires vasic-digital/containers as the sole orchestration layer. And it is **not** a submodule of this tree. The container work in this feature must resolve that before adding a bespoke stack — it is a prerequisite, not a detail.

WITHDRAWN 2026-09-01 — kept visible rather than quietly replaced, so a reader can tell a corrected fact from one that was never wrong.

- *Believed when written (2026-08-31):* no vasic-digital/containers gitlink was declared here, so §11.4.76(2) was an open prerequisite and the container work was blocked behind an operator decision.
- *Measured now (2026-09-01), and re-derivable in one line:*

```
git config -f .gitmodules --get-regexp containers
# submodule.submodules/containers.path submodules/containers
# submodule.submodules/containers.url git@github.com:vasic-
    digital/containers.git
ls submodules/containers/cmd # populated: boot/
    among others
git ls-tree HEAD submodules/containers # 160000 commit
    4dab992... submodules/containers
```

- *When it changed:* during this feature's own Phase 1 work, between the sentence above being written and this correction being made.
- *What still holds:* a bespoke Containerfile or compose-only stack is still forbidden by §11.4.76(4). The prerequisite is satisfied; the prohibition is not lifted.

The architecture is therefore **settled and not open**: the workshop is **containerised**, and it consumes submodules/containers — through its pkg/compose / pkg/health / pkg/runtime Go API, which is what §11.4.76(3) actually names, and **not** through its cmd/boot CLI, measured unusable for this workload. A thin workshop-boot adapter over that API plus thin bash wrappers over the adapter are *consumption* and so are

§11.4.76-legal; anything beyond thin wrapping must be contributed upstream. Do not read the struck-through text above as a live open question.

Evidence — where the numbers come from, and how to re-derive them

Where evidence is written

Per FR-040 and the Clarifications session, evidence goes to a **versioned directory inside the repository, retained with the commit that produced it** — never to transient logs. The repository’s existing practice is `_tests/evidence/<gate>/`, with raw instrument output kept alongside the verdict (for example `_tests/evidence/ally-audit/axe-json/` next to `run-audit.js`).

This feature follows the same convention under `workshop/evidence/<check>/`. The exact path is fixed by `contracts/`; this guide states the convention, not the authority.

Each check writes, at minimum: the **verdict** (0 / 1 / 2), the **raw instrument output**, the **command line** that produced it, and the **conditions** — host load average, whether an index build was in flight, which provider was configured. For the benchmarks, the **seed** as well, so the sample is re-drawable.

```
# CONTRACT ONLY. Confirms SC-018 after a run: every check left
  evidence behind.
ls -R workshop/evidence/
```

Success criterion proved: SC-018 — 100% of automated checks write evidence to the versioned location, verifiable by inspecting that location after a run.

How a reviewer re-derives every claimed number

No figure in this feature’s reports is to be taken on trust. Each one is re-derivable:

Claim	Re-derived by
transcript accuracy (SC-002)	re-draw the sample with the recorded seed, re-score against the retained blind human

Claim	Re-derived by
	reference using the frozen, hashed normaliser
coverage (SC-001)	re-run the span-union identity against <code>duration_s = 6928.75</code>
timestamp error (SC-003)	re-score the retained window onsets; also hand-check five passages against the reassembled video
suggest p95 (SC-005)	re-run <code>bench-suggest.sh</code> ; compare against the retained per-request latencies, not the summary
search p95 (SC-006)	re-run <code>bench-search.sh</code> and read the recorded load conditions — a number without them is not interpretable
retrieval quality (SC-007/008)	re-run the benchmark file; the zero-overlap subset is mechanically checkable from the file itself
citation soundness (SC-009)	machine: re-run L3 set membership. Human: follow ≥ 20 citations by hand. Both , per SC-009's wording
refusals (SC-010)	re-run the 10 adversarial questions at temperature 0
idempotency (FR-027)	re-run ingest twice, diff the content hashes
reference resolution (SC-016)	re-run resolution over the retained before/after snapshots
reassembly integrity (FR-007)	bash <code>workshop/scripts/extract-videos.sh</code> — RUNNABLE NOW

Mutation proofs — the half that is usually skipped

FR-032 and SC-012 require **every** automated check to carry a paired demonstration that it **fails** when its guarded condition is broken — 100%, no exceptions. A green run alone is not evidence; it is consistent with a check that cannot fail at all. §1.1 additionally requires the paired proof to include a real end-to-end run, not a stubbed one.

Runnable today — two existing scripts ship the mechanism, and running them is how a reviewer confirms the *pattern* works before the feature's own gates exist:

```
bash workshop/scripts/self-test.sh                # corrupts a
part; extractor MUST refuse
bash scripts/continuation-check.sh --prove-failure # seeds
violations; MUST report them
bash scripts/verify-governance-cascade.sh --prove-failure #
seeds 5 violations; rc=1
```

Verified 2026-09-01: --prove-failure exists on exactly two scripts under scripts/ — continuation-check.sh and verify-governance-cascade.sh. The cascade's proof catches 5 seeded violations as rc=1 and reports an **environment fault as rc=2** rather than accusing the tree — that three-valued outcome is the same discipline the preflight applies to itself, and it is worth reading before writing this feature's own proofs.

Owed by this feature — every one is BLOCKED: BUILD, and every one is listed so none is quietly dropped:

Check	Its mutation proof
coverage (SC-001)	delete a passage ⇒ gap reported, non-zero exit
accuracy scorer (SC-002)	corrupt 10% of the reference ⇒ WER moves comparably
timestamp scorer (SC-003)	shift a passage by 30 s ⇒ p95 breaches the 5 s bound
fresh-clone start (SC-004)	remove the reassembled video ⇒ start refuses, names what is missing
suggest bench (SC-005)	inject a 300 ms delay ⇒ p95 breaches 200 ms
search bench (SC-006)	run during a deliberate ingest ⇒ reserved capacity holds, or the failure is recorded

Check	Its mutation proof
retrieval bench (SC-007/008)	empty index generation $\Rightarrow$ 0%, non-zero exit
three-state search	feed Lumen's "No results found. \ Warning: Index is being updated..." string $\Rightarrow$ Degraded , never "no results"
citation check (SC-009)	cite a pid absent from the live generation $\Rightarrow$ refusal
refusal check (SC-010)	lower the margin threshold $\Rightarrow$ a near-miss question is answered, and the proof catches it
ingest idempotency (FR-027)	mint a fresh pid on re-ingest $\Rightarrow$ diff non-empty
reference resolution (SC-016)	key the resolver on content_hash $\Rightarrow$ <100%, non-zero exit
a11y audit (SC-017)	introduce a known Level-A violation $\Rightarrow$ axe reports it
evidence writing (SC-018)	suppress one check's evidence write $\Rightarrow$ the SC-018 check fails

What this guide does not prove

Stated explicitly, because a quickstart that implies more than it demonstrates is worse than one that names its gaps.

1. **No success criterion in US1 step 2 onward, US2, US3, US4 or US5 has been demonstrated.** Those components do not exist. Every command in those sections marked CONTRACT ONLY has **never been executed** and must not be quoted as a working invocation.
2. **The recording has never been listened to and no ASR has ever run on it** (U3). That it contains intelligible speech in a known language is an assumption. Every transcription estimate in this document rests on it.
3. **Transcription throughput on this CPU is unmeasured** (U2). The 40–80 minute idle figure — and roughly double under load — is arithmetic, not measurement.
4. **Idle embedding latency is unmeasured** (U4), because the lumen index --force rebuild has not finished. Until it does, SC-006 is neither supportable nor refutable.

5. **Chapter 1's media properties are not re-derivable on this host** — `ffprobe` is not installed (P1). Duration, codec and channel layout are carried from an earlier measurement, not reproduced.
6. **US4 under generation has no evidence at all.** There is no decoder on this host (P4). Config A proves the refusal plumbing; it does not prove a generator behaves.

Success criteria with no runnable validation devised here

Two, and both for the same honest reason:

- **SC-012** (“every automated check has a paired demonstration that it fails — 100%”) and **SC-013** (“the system distinguishes ‘unable to verify’ from ‘passed’ and ‘failed’ in 100% of its checks”) are **universally quantified over a set of checks that does not exist yet**. They cannot be validated by running something; they can only be validated by **enumerating the finished check inventory and confirming each entry has both properties**. The mutation-proof table above is that enumeration in its current, partial form — it is the seed of the SC-012 inventory, not a substitute for it.

What *can* be built, and what this guide recommends: a **meta-check** that walks the feature's check registry and fails when any registered check lacks either a paired `--prove-failure` mode or a three-valued verdict. That meta-check would make SC-012 and SC-013 mechanically verifiable — and it would itself need a paired mutation proof: register a check with no proof, and require the meta-check to catch it.

- **SC-015** (“the main repository and every submodule report a clean working tree”) is verifiable by `git status --porcelain` recursively, but it is a **release-time** condition, not a scenario outcome. It is deliberately not run from this guide: this guide performs no commits and no pushes, and the working tree was not clean while it was written.